Syncner Database Schema Documentation

📋 Table of Contents

1. Database Overview
2. Collection Schemas
3. Relationships
4. Indexes
5. Data Validation
6. Migration Scripts
7. Performance Optimization
8. Backup and Recovery

🗄️ Database Overview

Database Information

- **Database Type**: MongoDB (NoSQL Document Database)
- **Version**: MongoDB 6.0+
- **Hosting**: MongoDB Atlas (Cloud)
- **Database Name**: `syncner`
- **Connection**: MongoDB Atlas Connection String

Database Architecture

```
MongoDB Atlas Cluster
├── Primary Database (syncner)
│   ├── Collections (7 main collections)
│   ├── Indexes (Performance optimized)
│   └── Views (Aggregated data)
├── Backup Database
│   └── Automated daily backups
└── Analytics Database (Future)
    └── Read-only analytics data
```

Collection Overview

Collection	Purpose	Document Count (Est.)	Size (Est.)
`users`	User accounts and profiles	1,000-10,000	50MB-500MB
`curriculum`	Learning content and levels	100-1,000	10MB-100MB
`userprogress`	Student learning progress	1,000-10,000	100MB-1GB
`attendance`	Attendance records	10,000-100,000	50MB-500MB
`settings`	System configuration	1-10	1MB

Collection	Purpose	Document Count (Est.)	Size (Est.)
tracks	Career track definitions	5-20	1MB
uploads	File metadata	1,000-10,000	10MB-100MB

📊 Collection Schemas

1. Users Collection

Schema Definition

```
{
  _id: ObjectId,                    // Primary key
  name: String,                     // User's full name
  email: String,                    // Unique email address
  password: String,                 // Hashed password
  role: String,                     // 'student', 'staff', 'admin'
  careerTrack: String,              // Career track identifier
  college: String,                  // Educational institution
  department: String,               // Department within college
  phone: String,                    // Contact phone number
  location: String,                 // Geographic location
  gpa: String,                      // Grade point average
  graduationYear: String,           // Expected graduation year
  profileComplete: Boolean,         // Profile completion status
  isActive: Boolean,                // Account active status
  lastActive: Date,                 // Last login timestamp
  interests: [String],              // User interests array
  skills: [String],                 // User skills array
  createdAt: Date,                  // Account creation date
  updatedAt: Date                   // Last update timestamp
}
```

Example Document

```
{
  "_id": ObjectId("64f1a2b3c4d5e6f7g8h9i0j1"),
  "name": "John Doe",
  "email": "john.doe@example.com",
  "password":
"$2a$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/LewdBPj4J8K8K8K8K",
  "role": "student",
  "careerTrack": "full-stack",
  "college": "Example University",
  "department": "Computer Science",
  "phone": "+1-555-0123",
  "location": "New York, NY",
  "gpa": "3.8",
```

```
  "graduationYear": "2024",
  "profileComplete": true,
  "isActive": true,
  "lastActive": ISODate("2024-01-15T10:30:00Z"),
  "interests": ["web development", "machine learning"],
  "skills": ["JavaScript", "React", "Node.js"],
  "createdAt": ISODate("2024-01-01T00:00:00Z"),
  "updatedAt": ISODate("2024-01-15T10:30:00Z")
}
```

Validation Rules

```
{
  name: { required: true, minLength: 2, maxLength: 100 },
  email: { required: true, unique: true, format: 'email' },
  password: { required: true, minLength: 8 },
  role: { required: true, enum: ['student', 'staff', 'admin'] },
  careerTrack: { required: true, enum: ['full-stack', 'ai-ml', 'ui-ux',
'data-science'] },
  profileComplete: { type: Boolean, default: false },
  isActive: { type: Boolean, default: true }
}
```

2. Curriculum Collection

Schema Definition

```
{
  _id: ObjectId,                    // Primary key
  title: String,                    // Level title
  description: String,              // Level description
  careerTrack: String,              // Associated career track
  order: Number,                    // Display order
  difficulty: String,               // 'beginner', 'intermediate',
'advanced'
  duration: String,                 // Estimated duration
  estimatedHours: Number,           // Estimated hours to complete
  skills: [String],                 // Skills taught in this level
  prerequisites: [String],          // Prerequisite skills
  isActive: Boolean,                // Level active status
  readFirst: {                      // Introduction content
    brief: String,                  // Brief overview
    whyLearn: String,               // Why learn this topic
    whereUsed: String,              // Where it's used in industry
    customExample: String           // Custom example
  },
  learn: [{                         // Learning materials
    id: String,                     // Unique identifier
```

```
    title: String,                  // Material title
    content: String,                // Material content
    externalLink: String,           // External resource link
    order: Number                   // Display order
  }],
  test: [{                          // Test questions
    id: String,                     // Unique identifier
    question: String,               // Question text
    options: [String],              // Answer options
    answer: String,                 // Correct answer
    order: Number                   // Display order
  }],
  exercise: [{                      // Exercise tasks
    id: String,                     // Unique identifier
    title: String,                  // Exercise title
    description: String,            // Exercise description
    resources: [String],            // Resource links
    order: Number                   // Display order
  }],
  createdAt: Date,                  // Creation timestamp
  updatedAt: Date                   // Last update timestamp
}
```

Example Document

```
{
  "_id": ObjectId("64f1a2b3c4d5e6f7g8h9i0j2"),
  "title": "Introduction to React",
  "description": "Learn the fundamentals of React library",
  "careerTrack": "full-stack",
  "order": 3,
  "difficulty": "beginner",
  "duration": "2 weeks",
  "estimatedHours": 15,
  "skills": ["React", "JavaScript", "JSX"],
  "prerequisites": ["HTML", "CSS", "JavaScript"],
  "isActive": true,
  "readFirst": {
    "brief": "React is a JavaScript library for building user interfaces",
    "whyLearn": "React is widely used in modern web development",
    "whereUsed": "Used by Facebook, Netflix, Airbnb, and many others",
    "customExample": "Building a simple todo application"
  },
  "learn": [
    {
      "id": "learn-1",
      "title": "React Components",
      "content": "Learn about React components and their lifecycle",
      "externalLink": "https://reactjs.org/docs/components-and-
props.html",
      "order": 1
```

```
      }
    ],
    "test": [
      {
        "id": "test-1",
        "question": "What is a React component?",
        "options": [
          "A JavaScript function",
          "A CSS class",
          "An HTML element",
          "A database table"
        ],
        "answer": "A JavaScript function",
        "order": 1
      }
    ],
    "exercise": [
      {
        "id": "exercise-1",
        "title": "Create a React Component",
        "description": "Build a simple React component that displays your
name",
        "resources": ["https://reactjs.org/tutorial/tutorial.html"],
        "order": 1
      }
    ],
    "createdAt": ISODate("2024-01-01T00:00:00Z"),
    "updatedAt": ISODate("2024-01-15T10:30:00Z")
}
```

3. UserProgress Collection

Schema Definition

```
{
  _id: ObjectId,                    // Primary key
  userId: ObjectId,                 // Reference to User
  careerTrack: String,              // Career track identifier
  currentLevel: Number,             // Current level number
  totalPoints: Number,              // Total points earned
  streakDays: Number,               // Current streak
  completedLevels: [{               // Completed levels array
    levelId: ObjectId,              // Reference to Curriculum
    levelOrder: Number,             // Level order number
    completedAt: Date,              // Completion timestamp
    completedSections: {            // Section completion status
      learn: Boolean,               // Learn section completed
      test: Boolean,                // Test section completed
      exercise: Boolean             // Exercise section completed
    },
    learnProgress: {                // Learn section progress
```

```
      timeSpent: Number,              // Time spent in seconds
      linksAccessed: [String],        // Links accessed
      completed: Boolean,             // Completion status
      startTime: Date,                // Start timestamp
      endTime: Date                   // End timestamp
    },
    testProgress: {                   // Test section progress
      score: Number,                  // Test score (0-100)
      attempts: Number,               // Number of attempts
      completed: Boolean,             // Completion status
      startTime: Date,                // Start timestamp
      endTime: Date,                  // End timestamp
      answers: [{                     // Answer history
        questionId: String,           // Question identifier
        selectedAnswer: String,       // Selected answer
        isCorrect: Boolean,           // Correctness
        attemptNumber: Number         // Attempt number
      }]
    },
    exerciseProgress: {               // Exercise section progress
      submissions: [{                 // Submission history
        links: [String],              // Submission links
        submittedAt: Date,            // Submission timestamp
        status: String,               // 'submitted', 'approved', 'rejected'
        feedback: String,             // Staff feedback
        score: Number                 // Exercise score
      }],
      completed: Boolean,             // Completion status
      startTime: Date,                // Start timestamp
      endTime: Date                   // End timestamp
    }
  }],
  createdAt: Date,                    // Creation timestamp
  updatedAt: Date                     // Last update timestamp
}
```

Example Document

```
{
  "_id": ObjectId("64f1a2b3c4d5e6f7g8h9i0j3"),
  "userId": ObjectId("64f1a2b3c4d5e6f7g8h9i0j1"),
  "careerTrack": "full-stack",
  "currentLevel": 3,
  "totalPoints": 1250,
  "streakDays": 5,
  "completedLevels": [
    {
      "levelId": ObjectId("64f1a2b3c4d5e6f7g8h9i0j2"),
      "levelOrder": 1,
      "completedAt": ISODate("2024-01-10T15:30:00Z"),
      "completedSections": {
```

```
          "learn": true,
          "test": true,
          "exercise": true
        },
        "learnProgress": {
          "timeSpent": 1200,
          "linksAccessed": ["https://reactjs.org/docs/components-and-
  props.html"],
          "completed": true,
          "startTime": ISODate("2024-01-08T09:00:00Z"),
          "endTime": ISODate("2024-01-10T15:30:00Z")
        },
        "testProgress": {
          "score": 85,
          "attempts": 2,
          "completed": true,
          "startTime": ISODate("2024-01-10T16:00:00Z"),
          "endTime": ISODate("2024-01-10T16:45:00Z"),
          "answers": [
            {
              "questionId": "test-1",
              "selectedAnswer": "A JavaScript function",
              "isCorrect": true,
              "attemptNumber": 1
            }
          ]
        },
        "exerciseProgress": {
          "submissions": [
            {
              "links": ["https://github.com/johndoe/react-component"],
              "submittedAt": ISODate("2024-01-12T14:00:00Z"),
              "status": "approved",
              "feedback": "Great work! Well-structured component.",
              "score": 90
            }
          ],
          "completed": true,
          "startTime": ISODate("2024-01-11T10:00:00Z"),
          "endTime": ISODate("2024-01-12T14:00:00Z")
        }
      }
    ],
    "createdAt": ISODate("2024-01-01T00:00:00Z"),
    "updatedAt": ISODate("2024-01-15T10:30:00Z")
  }
```

4. Attendance Collection

Schema Definition

```
{
  _id: ObjectId,                    // Primary key
  userId: ObjectId,                 // Reference to User
  date: Date,                       // Attendance date
  status: String,                   // 'present', 'absent', 'late'
  markedAt: Date,                   // Marking timestamp
  markedBy: ObjectId,               // Who marked attendance (staff/admin)
  notes: String,                    // Additional notes
  createdAt: Date,                  // Creation timestamp
  updatedAt: Date                   // Last update timestamp
}
```

Example Document

```
{
  "_id": ObjectId("64f1a2b3c4d5e6f7g8h9i0j4"),
  "userId": ObjectId("64f1a2b3c4d5e6f7g8h9i0j1"),
  "date": ISODate("2024-01-15T00:00:00Z"),
  "status": "present",
  "markedAt": ISODate("2024-01-15T09:15:00Z"),
  "markedBy": ObjectId("64f1a2b3c4d5e6f7g8h9i0j5"),
  "notes": "Attended all sessions",
  "createdAt": ISODate("2024-01-15T09:15:00Z"),
  "updatedAt": ISODate("2024-01-15T09:15:00Z")
}
```

5. Settings Collection

Schema Definition

```
{
  _id: ObjectId,                    // Primary key
  category: String,                 // Settings category
  key: String,                      // Setting key
  value: Mixed,                     // Setting value
  description: String,              // Setting description
  isActive: Boolean,                // Setting active status
  updatedBy: ObjectId,              // Who updated the setting
  createdAt: Date,                  // Creation timestamp
  updatedAt: Date                   // Last update timestamp
}
```

Example Document

```
{
  "_id": ObjectId("64f1a2b3c4d5e6f7g8h9i0j6"),
  "category": "ai",
  "key": "gemini_api_key",
  "value": "configured",
  "description": "Google Gemini API key for AI chat functionality",
  "isActive": true,
  "updatedBy": ObjectId("64f1a2b3c4d5e6f7g8h9i0j7"),
  "createdAt": ISODate("2024-01-01T00:00:00Z"),
  "updatedAt": ISODate("2024-01-15T10:30:00Z")
}
```

6. Tracks Collection

Schema Definition

```
{
  _id: ObjectId,                    // Primary key
  name: String,                     // Track name
  displayName: String,              // Display name
  description: String,              // Track description
  totalLevels: Number,              // Total number of levels
  difficulty: String,               // Overall difficulty
  duration: String,                 // Total duration
  skills: [String],                 // Skills covered
  careerPaths: [String],            // Career paths
  averageSalary: String,            // Average salary range
  isActive: Boolean,                // Track active status
  createdAt: Date,                  // Creation timestamp
  updatedAt: Date                   // Last update timestamp
}
```

Example Document

```
{
  "_id": ObjectId("64f1a2b3c4d5e6f7g8h9i0j8"),
  "name": "full-stack",
  "displayName": "Full Stack Development",
  "description": "Learn both frontend and backend development",
  "totalLevels": 10,
  "difficulty": "intermediate",
  "duration": "6 months",
  "skills": ["HTML", "CSS", "JavaScript", "React", "Node.js", "MongoDB"],
  "careerPaths": ["Frontend Developer", "Backend Developer", "Full Stack
Developer"],
  "averageSalary": "$70,000 - $120,000",
  "isActive": true,
  "createdAt": ISODate("2024-01-01T00:00:00Z"),
```

```
  "updatedAt": ISODate("2024-01-15T10:30:00Z")
}
```

7. Uploads Collection

Schema Definition

```
{
  _id: ObjectId,                    // Primary key
  userId: ObjectId,                 // Reference to User
  originalName: String,             // Original filename
  filename: String,                 // Stored filename
  mimetype: String,                 // File MIME type
  size: Number,                     // File size in bytes
  path: String,                     // File storage path
  url: String,                      // File access URL
  uploadedAt: Date,                 // Upload timestamp
  isActive: Boolean,                // File active status
  createdAt: Date,                  // Creation timestamp
  updatedAt: Date                   // Last update timestamp
}
```

Example Document

```
{
  "_id": ObjectId("64f1a2b3c4d5e6f7g8h9i0j9"),
  "userId": ObjectId("64f1a2b3c4d5e6f7g8h9i0j1"),
  "originalName": "project-submission.zip",
  "filename": "1751491335517-746469234-project-submission.zip",
  "mimetype": "application/zip",
  "size": 2048576,
  "path": "./uploads/1751491335517-746469234-project-submission.zip",
  "url": "/uploads/1751491335517-746469234-project-submission.zip",
  "uploadedAt": ISODate("2024-01-15T14:30:00Z"),
  "isActive": true,
  "createdAt": ISODate("2024-01-15T14:30:00Z"),
  "updatedAt": ISODate("2024-01-15T14:30:00Z")
}
```

🔗 Relationships

Entity Relationship Diagram

```
Users (1) ──────── (1) UserProgress
  │                     │
  │                     │
```

```
    │ (1) ──────────────── (M) Attendance
    │
    │ (1) ──────────────── (M) Uploads
    │
    │ (M) ──────────────── (1) Curriculum
    │
    │ (M) ──────────────── (1) Tracks

  Curriculum (1) ──────── (M) UserProgress
    │
    │ (1) ──────────────── (1) Tracks

  Settings (Independent)
```

Relationship Details

1. User ↔ UserProgress (1:1)

- **Relationship**: One user has one progress record per career track
- **Foreign Key**: `userProgress.userId` → `users._id`
- **Index**: `{ userId: 1, careerTrack: 1 }`

2. User ↔ Attendance (1:M)

- **Relationship**: One user can have multiple attendance records
- **Foreign Key**: `attendance.userId` → `users._id`
- **Index**: `{ userId: 1, date: 1 }`

3. User ↔ Uploads (1:M)

- **Relationship**: One user can upload multiple files
- **Foreign Key**: `uploads.userId` → `users._id`
- **Index**: `{ userId: 1, uploadedAt: -1 }`

4. Curriculum ↔ UserProgress (1:M)

- **Relationship**: One curriculum level can be in multiple user progress records
- **Foreign Key**: `userProgress.completedLevels.levelId` → `curriculum._id`
- **Index**: `{ "completedLevels.levelId": 1 }`

5. Tracks ↔ Curriculum (1:M)

- **Relationship**: One track can have multiple curriculum levels
- **Foreign Key**: `curriculum.careerTrack` → `tracks.name`
- **Index**: `{ careerTrack: 1, order: 1 }`

📈 Indexes

Performance Indexes

1. Users Collection

```
// Unique email index
db.users.createIndex({ email: 1 }, { unique: true });

// Career track and role index
db.users.createIndex({ careerTrack: 1, role: 1 });

// Active users index
db.users.createIndex({ isActive: 1, lastActive: -1 });

// Search index
db.users.createIndex({
  name: "text",
  email: "text",
  college: "text"
});
```

2. UserProgress Collection

```
// User and career track index
db.userprogress.createIndex({ userId: 1, careerTrack: 1 });

// Level completion index
db.userprogress.createIndex({
  "completedLevels.levelId": 1,
  "completedLevels.levelOrder": 1
});

// Progress tracking index
db.userprogress.createIndex({
  userId: 1,
  currentLevel: 1
});
```

3. Curriculum Collection

```
// Career track and order index
db.curriculum.createIndex({ careerTrack: 1, order: 1 });

// Active curriculum index
db.curriculum.createIndex({ isActive: 1, careerTrack: 1 });

// Difficulty index
db.curriculum.createIndex({ difficulty: 1, careerTrack: 1 });
```

4. Attendance Collection

```
// User and date index
db.attendance.createIndex({ userId: 1, date: 1 });

// Date range index
db.attendance.createIndex({ date: -1 });

// Status index
db.attendance.createIndex({ status: 1, date: -1 });
```

5. Settings Collection

```
// Category and key index
db.settings.createIndex({ category: 1, key: 1 });

// Active settings index
db.settings.createIndex({ isActive: 1, category: 1 });
```

Compound Indexes

```
// User progress analytics
db.userprogress.createIndex({
  careerTrack: 1,
  currentLevel: 1,
  totalPoints: -1
});

// Attendance analytics
db.attendance.createIndex({
  userId: 1,
  status: 1,
  date: -1
});

// Curriculum search
db.curriculum.createIndex({
  careerTrack: 1,
  difficulty: 1,
  isActive: 1
});
```

✅ Data Validation

Mongoose Schema Validation

1. User Schema Validation

```
const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    minlength: [2, 'Name must be at least 2 characters'],
    maxlength: [100, 'Name cannot exceed 100 characters'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    match: [/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/, 'Please enter a
valid email']
  },
  password: {
    type: String,
    required: [true, 'Password is required'],
    minlength: [8, 'Password must be at least 8 characters'],
    select: false // Don't include in queries by default
  },
  role: {
    type: String,
    required: [true, 'Role is required'],
    enum: {
      values: ['student', 'staff', 'admin'],
      message: 'Role must be student, staff, or admin'
    }
  },
  careerTrack: {
    type: String,
    required: [true, 'Career track is required'],
    enum: {
      values: ['full-stack', 'ai-ml', 'ui-ux', 'data-science'],
      message: 'Invalid career track'
    }
  }
}, {
  timestamps: true,
  toJSON: { virtuals: true },
  toObject: { virtuals: true }
});
```

2. Curriculum Schema Validation

```
const curriculumSchema = new mongoose.Schema({
  title: {
```

```
    type: String,
    required: [true, 'Title is required'],
    maxlength: [200, 'Title cannot exceed 200 characters']
  },
  order: {
    type: Number,
    required: [true, 'Order is required'],
    min: [1, 'Order must be at least 1']
  },
  difficulty: {
    type: String,
    required: [true, 'Difficulty is required'],
    enum: {
      values: ['beginner', 'intermediate', 'advanced'],
      message: 'Difficulty must be beginner, intermediate, or advanced'
    }
  },
  estimatedHours: {
    type: Number,
    required: [true, 'Estimated hours is required'],
    min: [1, 'Estimated hours must be at least 1']
  }
});
```

Custom Validation Functions

```
// Custom validator for unique order within career track
curriculumSchema.pre('save', async function(next) {
  const existingLevel = await this.constructor.findOne({
    careerTrack: this.careerTrack,
    order: this.order,
    _id: { $ne: this._id }
  });

  if (existingLevel) {
    throw new Error('Order must be unique within career track');
  }

  next();
});

// Custom validator for password strength
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();

  // Check password strength
  const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-
Za-z\d@$!%*?&]/;
  if (!passwordRegex.test(this.password)) {
    throw new Error('Password must contain at least one uppercase letter,
one lowercase letter, one number, and one special character');
```

```
  }

  next();
});
```

🔄 Migration Scripts

1. Initial Database Setup

```
// migrations/initialSetup.js
const mongoose = require('mongoose');
const User = require('../models/User');
const Curriculum = require('../models/Curriculum');
const Track = require('../models/Track');

const initialSetup = async () => {
  try {
    // Create default tracks
    const tracks = [
      {
        name: 'full-stack',
        displayName: 'Full Stack Development',
        description: 'Learn both frontend and backend development',
        totalLevels: 10,
        difficulty: 'intermediate',
        duration: '6 months',
        skills: ['HTML', 'CSS', 'JavaScript', 'React', 'Node.js',
'MongoDB'],
        careerPaths: ['Frontend Developer', 'Backend Developer', 'Full
Stack Developer'],
        averageSalary: '$70,000 - $120,000',
        isActive: true
      },
      {
        name: 'ai-ml',
        displayName: 'AI & Machine Learning',
        description: 'Learn artificial intelligence and machine learning',
        totalLevels: 12,
        difficulty: 'advanced',
        duration: '8 months',
        skills: ['Python', 'TensorFlow', 'PyTorch', 'Data Science',
'Statistics'],
        careerPaths: ['AI Engineer', 'Data Scientist', 'ML Engineer'],
        averageSalary: '$90,000 - $150,000',
        isActive: true
      }
    ];

    await Track.insertMany(tracks);
    console.log('Default tracks created');
```

```
    // Create default admin user
    const adminUser = {
      name: 'System Administrator',
      email: 'admin@syncner.com',
      password: 'Admin123!@#',
      role: 'admin',
      careerTrack: 'full-stack',
      profileComplete: true,
      isActive: true
    };

    await User.create(adminUser);
    console.log('Default admin user created');

  } catch (error) {
    console.error('Initial setup failed:', error);
  }
};

module.exports = initialSetup;
```

2. Data Migration Script

```
// migrations/migrateUserProgress.js
const mongoose = require('mongoose');
const UserProgress = require('../models/UserProgress');

const migrateUserProgress = async () => {
  try {
    // Find users without progress records
    const usersWithoutProgress = await User.find({
      role: 'student',
      _id: { $nin: await UserProgress.distinct('userId') }
    });

    // Create progress records for users
    const progressRecords = usersWithoutProgress.map(user => ({
      userId: user._id,
      careerTrack: user.careerTrack,
      currentLevel: 1,
      totalPoints: 0,
      streakDays: 0,
      completedLevels: []
    }));

    await UserProgress.insertMany(progressRecords);
    console.log(`Created ${progressRecords.length} progress records`);

  } catch (error) {
    console.error('Migration failed:', error);
  }
```

```
};

module.exports = migrateUserProgress;
```

3. Index Creation Script

```
// migrations/createIndexes.js
const mongoose = require('mongoose');

const createIndexes = async () => {
  try {
    const db = mongoose.connection.db;

    // Users indexes
    await db.collection('users').createIndex({ email: 1 }, { unique: true
});
    await db.collection('users').createIndex({ careerTrack: 1, role: 1 });
    await db.collection('users').createIndex({ isActive: 1, lastActive: -1
});

    // UserProgress indexes
    await db.collection('userprogress').createIndex({ userId: 1,
careerTrack: 1 });
    await db.collection('userprogress').createIndex({
      "completedLevels.levelId": 1,
      "completedLevels.levelOrder": 1
    });

    // Curriculum indexes
    await db.collection('curriculum').createIndex({ careerTrack: 1, order:
1 });
    await db.collection('curriculum').createIndex({ isActive: 1,
careerTrack: 1 });

    // Attendance indexes
    await db.collection('attendance').createIndex({ userId: 1, date: 1 });
    await db.collection('attendance').createIndex({ date: -1 });

    console.log('All indexes created successfully');

  } catch (error) {
    console.error('Index creation failed:', error);
  }
};

module.exports = createIndexes;
```

⚡ Performance Optimization

Query Optimization

1. Efficient Queries

```
// Good: Use projection to limit fields
const users = await User.find({ role: 'student' })
  .select('name email careerTrack lastActive')
  .lean();

// Good: Use aggregation for complex queries
const studentProgress = await UserProgress.aggregate([
  { $match: { careerTrack: 'full-stack' } },
  { $lookup: {
    from: 'users',
    localField: 'userId',
    foreignField: '_id',
    as: 'user'
  }},
  { $unwind: '$user' },
  { $project: {
    'user.name': 1,
    'user.email': 1,
    currentLevel: 1,
    totalPoints: 1
  }}
]);

// Good: Use pagination for large datasets
const students = await User.find({ role: 'student' })
  .skip((page - 1) * limit)
  .limit(limit)
  .sort({ createdAt: -1 });
```

2. Caching Strategy

```
const NodeCache = require('node-cache');
const cache = new NodeCache({ stdTTL: 600 }); // 10 minutes

// Cache frequently accessed data
const getCachedCurriculum = async (careerTrack) => {
  const cacheKey = `curriculum_${careerTrack}`;
  let curriculum = cache.get(cacheKey);

  if (!curriculum) {
    curriculum = await Curriculum.find({
      careerTrack,
      isActive: true
    }).sort({ order: 1 });
    cache.set(cacheKey, curriculum);
  }
```

```
  return curriculum;
};
```

Database Monitoring

```
// Performance monitoring
const monitorPerformance = () => {
  mongoose.connection.on('connected', () => {
    console.log('MongoDB connected successfully');
  });

  mongoose.connection.on('error', (err) => {
    console.error('MongoDB connection error:', err);
  });

  // Monitor slow queries
  mongoose.set('debug', (collectionName, method, query, doc) => {
    console.log(`${collectionName}.${method}`, query);
  });
};
```

💾 Backup and Recovery

Backup Strategy

```
// Automated backup script
const backupDatabase = async () => {
  const { exec } = require('child_process');
  const date = new Date().toISOString().split('T')[0];

  const command = `mongodump --uri="${process.env.MONGODB_URI}" --
out="./backups/backup_${date}"`;

  exec(command, (error, stdout, stderr) => {
    if (error) {
      console.error('Backup failed:', error);
      return;
    }
    console.log('Backup completed successfully');
  });
};

// Schedule daily backups
const cron = require('node-cron');
cron.schedule('0 2 * * *', () => {
  backupDatabase();
});
```

Recovery Procedures

```
// Database recovery script
const restoreDatabase = async (backupDate) => {
  const { exec } = require('child_process');

  const command = `mongorestore --uri="${process.env.MONGODB_URI}"
./backups/backup_${backupDate}`;

  exec(command, (error, stdout, stderr) => {
    if (error) {
      console.error('Restore failed:', error);
      return;
    }
    console.log('Database restored successfully');
  });
};
```

Data Integrity Checks

```
// Data integrity validation
const validateDataIntegrity = async () => {
  try {
    // Check for orphaned records
    const orphanedProgress = await UserProgress.find({
      userId: { $nin: await User.distinct('_id') }
    });

    if (orphanedProgress.length > 0) {
      console.warn(`Found ${orphanedProgress.length} orphaned progress
records`);
    }

    // Check for missing progress records
    const usersWithoutProgress = await User.find({
      role: 'student',
      _id: { $nin: await UserProgress.distinct('userId') }
    });

    if (usersWithoutProgress.length > 0) {
      console.warn(`Found ${usersWithoutProgress.length} users without
progress records`);
    }

    console.log('Data integrity check completed');

  } catch (error) {
    console.error('Data integrity check failed:', error);
  }
};
```

Database Schema Version: v1.0
Submitted On: September 2025
MongoDB Version: 6.0+